Reconstructing OpenAI's nearly quartic query separation

Comparing candidates and predicting a hidden address

Will Cook · November 2026

Abstract. OpenAI proved that the optimal exponent bounding randomized query complexity in terms of quantum query complexity for total Boolean functions is four. The quantum algorithm compares columns before verifying certificates. A comparison between two losing columns can have any bias. Give each column a weight equal to its probability of winning fresh comparisons against all previous pivots. When the next pivot is sampled proportionally to these weights, the bias of each pair cancels and the expected total weight falls by at least half. For the lower bound, a simulator with a fixed query budget predicts independent Boolean values from the cell address in a classical query for one certificate bit.

This companion gives the construction and both query bounds for readers familiar with query complexity and quantum search; prior reading of OpenAI's paper is not required. We derive the sampling rule by cancelling arbitrary comparison biases and use a capped predictor to avoid conditioning on an adaptively chosen query count. The worked exposition develops the operations through small examples; the full proof reconstruction supplies the supporting derivations, including the Forrelation lower bound used here.

1. The fourth power

Write $R(f)$ and $Q(f)$ for the bounded-error randomized and quantum query complexities of a total Boolean function f. A classical query reads one input bit. A quantum algorithm may query a superposition of bit indices. Aaronson, Ben-David, Kothari, Rao and Tal proved that $O(Q(f)^4)$ queries suffice even for a deterministic algorithm and conjectured $R(f) = O(Q(f)^3)$ [1, Conjecture 2]. Earlier cheat-sheet constructions attain powers approaching three [2–4]. OpenAI [5] showed that the quartic bound is sharp in its exponent.

Arrange m strings of m bits in columns, as in earlier pointer functions [6; 7]. A winning column has an all-one string and a certificate giving the row of a zero in every other string. Each column has its own array of certificates. As in the cheat-sheet construction [2], values of a partial Boolean function P on additional strings form the certificate address.

Take P to be k-fold Forrelation on k blocks of m bits. At fixed k a quantum algorithm decodes the address on promised inputs with bounded error in polylogarithmically many queries. To compare two columns, read one bit of each string at the row stored in the other selected cell. If the winning address is decoded correctly, the bit read in the other string is zero. The bit read in the winning string is one even if the other address is wrong. These comparisons yield a short list in $\widetilde{O}_k(\sqrt{m})$ queries, suppressing logarithmic factors. For each candidate, search for a zero in its string, a pointer to a one or an inconsistency in its circuit transcript. The circuit for the promise and value of P has $\widetilde{O}_k(m)$ gates, so verification costs another $\widetilde{O}_k(\sqrt{m})$ queries.

Computing P on promised inputs requires $\widetilde{\Omega}_k(m^{1-1/k})$ randomized queries [3]. A classical algorithm querying the winning certificate must specify its entire address. A uniform choice of winning column gives a further factor m. Up to logarithms, the quantum upper and randomized lower bounds are $(m, m^{3-1/k})$ for balanced AND–OR cheat sheets using the same P [2, § 3], and $(\sqrt{m}, m^{2-1/k})$ here. In powers of $\sqrt{m}$ the new lower bound has exponent $4-2/k$. Already $k = 3$ rules out the cubic conjecture.

Theorem 1 (OpenAI [5, Theorem 1.1], with explicit bounds from this reconstruction). *For each integer $k \geq 2$ there are total Boolean functions $F_{k,m}$ indexed by powers of four $m \geq 4$ such that, for a constant C_k and $\ell = \log_2 m$, the quantum bound below holds for every such m and the randomized bound holds for all sufficiently large m at fixed k:*

$$Q(F_{k,m}) \leq C_k \sqrt{m}\, \ell^6, \qquad R(F_{k,m}) \geq \frac{m^{2-1/k}}{400\ell^2}. \tag{1}$$

The optimal universal exponent in $R(f) = O((1+Q(f))^\alpha)$ for total Boolean f is four.[1]

2. The certificate

In addition to its string $X_i \in \{0,1\}^m$, column $i \in [m]$ holds $r = 20 \log_2 m$ inputs $A_{i,1}, \ldots, A_{i,r}$ to P and $2^r = m^{20}$ certificate cells $B_{i,s}$, one for each address $s \in \{0,1\}^r$.

The verifier must also check that the address inputs satisfy the promise and that their values give the selected address. Fix an exact fan-in-two Boolean circuit $\mathcal{C}$ with two outputs, a domain bit and the value of P on its domain. Each cell contains a proposed evaluation of $\mathcal{C}$ for each address input, one bit per non-input gate, and one ℓ-bit row pointer $p_{i,s,j}$ into each column $j \neq i$. Since $m = 2^\ell$, every pointer word represents a row.

Column i **wins** if $X_i = 1^m$, all its address inputs belong to the domain of P, and the cell with label

$$s_i = (P(A_{i,1}), \ldots, P(A_{i,r}))$$

contains the true circuit evaluations and $X_j[p_{i,s_i,j}] = 0$ for every $j \neq i$. The function equals one if a column wins and zero otherwise. This defines its value on every input, including those with off-promise address inputs. Two distinct columns cannot both win, since the pointer from one would have to specify a zero in the other's all-one string.

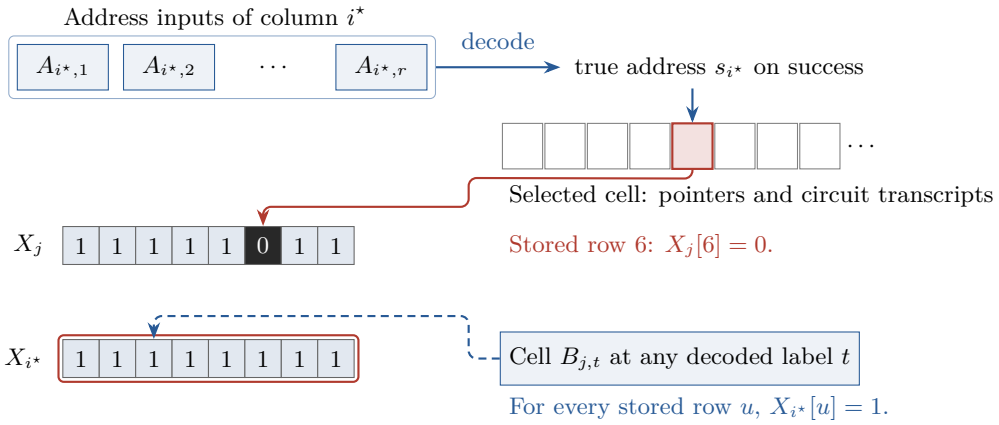


Figure 1: A comparison with the winning column $i^\star$. From the true cell of $i^\star$ the algorithm obtains row 6 and reads $X_j[6] = 0$. At any decoded address t for j, the reverse pointer stores a row u with $X_{i^\star}[u] = 1$.

To verify column i at a proposed address s, test that $X_i = 1^m$ and that $X_j[p_{i,s,j}] = 0$ for every $j \neq i$. In the proposed evaluation of $\mathcal{C}$ on $A_{i,h}$ (the *transcript*), compare each gate bit to the gate applied to its inputs, and the two output bits to $(1, s_h)$. By induction in circuit order, a transcript consistent at every gate equals the true evaluation. A false transcript therefore disagrees at some gate. A true transcript for an off-promise input or an incorrect address bit

[1] We use worst-case bit-query caps and error at most 1/3. For partial functions the cap applies on every input and branch, the error bound on the promise.

disagrees at an output. For a losing column at least one test fails at *every* address.[2] There are

$$K = m + (m-1) + r(S+2) \tag{2}$$

tests, where S counts non-input gates. Checking a gate value and its two inputs costs at most three queries, and reading a pointed-to bit costs $\ell+1$. We use this circuit for both bounds, since its gate list determines the cell format.

For k-fold Forrelation (§5) we use the following bounds [5, Lemma 2.1]. A reversible decoder for P with error η on its domain costs $O_k(1+\log(1/\eta))$ queries on every input. An exact circuit as above has $S = O_k(m\ell^2)$, so $K = O_k(m\ell^3)$ and the input length is polynomial in m at fixed k. For the classical lower bound, take L below the randomized query complexity of P. For fixed k and all sufficiently large m we may choose

$$L := \left\lfloor m^{1-1/k}/\ell^2 \right\rfloor < R(P). \tag{3}$$

3. Finding a winner by comparison

3.1 Comparing two columns

For $i \neq j$, decode addresses s for i and t for j. Read $X_j[p_{i,s,j}]$ and $X_i[p_{j,t,i}]$. If exactly one of these bits is zero, select the column storing the pointer to that zero. Break ties by smaller index. Suppose $i = i^\star$ is the winning column. If the algorithm decodes its address correctly, it reads a zero in X_j and a one in $X_{i^\star} = 1^m$. It therefore selects $i^\star$ regardless of the decoded address for j, even when none of j's address inputs satisfies the promise.

Write ε for the allowed error in decoding a whole address on promised inputs. Use error ε/r for each of its r bits. Use the same coherent circuit for both orderings of an unordered pair, retaining both decoded addresses. Write a_{ij} for the probability of selecting i in this comparison, and set $a_{ii} = 0$. On a fixed input the two possible outcomes of one comparison give

$$a_{ij} + a_{ji} = 1 \quad (i \neq j), \qquad a_{ii} = 0. \tag{4}$$

Thus $i^\star$ is *dominant* in the sense that $a_{i^\star j} \geq 1-\varepsilon$ for every $j \neq i^\star$. For a pair of losing columns either selection probability may lie anywhere in $[0,1]$. One comparison or its inverse uses $c = O_k(r\log(r/\varepsilon)+\ell)$ queries.

3.2 A tournament with probabilistic comparisons

The algorithm samples a sequence of indices, called pivots, and returns them as its candidate list [5, Proposition 3.2]. First suppose we know the comparison probabilities. We seek a sampling rule for which the list is short and contains the dominant column with high probability.

For deterministic comparisons one can retain the candidates selected over every previous pivot and draw uniformly from this set [8, § 6.1]. With probabilistic comparisons, weight each candidate by the probability of selecting it over every previous pivot in fresh, independent comparisons. After pivots $\rho_1, \ldots, \rho_n$, the individual and total weights are

$$w_i = \prod_{h=1}^{n} a_{i\rho_h}, \qquad W = \sum_i w_i.$$

Initially all weights are one. After drawing j, replace each w_i by $w_i a_{ij}$, so the new weight of j is zero. Suppose the probability of drawing j is q_j. The contribution of an unordered pair i, j to the expected remaining weight is $w_i q_j a_{ij} + w_j q_i a_{ji}$. For uniform sampling the two coefficients can

[2] The off-promise example isolates the domain test: an all-one column has correct gate values and pointers to zeros, yet its address is undefined.

differ. By (4), independence from an arbitrary comparison bias requires $w_i q_j = w_j q_i$. Drawing j with probability w_j/W gives equality for every pair.[3] Hence, while $W > 0$,

$$\mathbb{E}[W_{\text{new}} \mid w] = \frac{1}{W} \sum_{i,j} w_i w_j a_{ij} = \frac{W^2 - \sum_i w_i^2}{2W} \leq \frac{W}{2}. \tag{5}$$

Equivalently, in two independent draws from the weights, the ordered pairs (i, j) and (j, i) have equal probability. For $i \neq j$ the two selection probabilities sum to one, and for $i = j$ the contribution is zero.

Stop if $W = 0$ and keep it zero thereafter. After T steps the expected weight is at most $m2^{-T}$. If the dominant column is still unselected, its weight is at least $(1 - \varepsilon)^T \geq 1 - T\varepsilon$. With $T = \lceil \log_2(1000m) \rceil$ and $\varepsilon = 1/(1000T)$ this lower bound exceeds $1/2$. By Markov's inequality the probability that the column is absent from the list is therefore at most $2m2^{-T}$.

The quantum algorithm has only the reversible comparison circuit and the list of pivots drawn so far. To draw its next pivot, it prepares a uniform superposition of i and compares i with every listed pivot in fresh workspaces, coherently controlled by the candidate indices. If $|\phi(i, \rho_b)\rangle$ is the entire state of one comparison, the preparation produces

$$\frac{1}{\sqrt{m}} \sum_i |i\rangle \bigotimes_{b=1}^{n} |\phi(i, \rho_b)\rangle.$$

Set a flag if every comparison selects i. For a fixed i the comparison states form a tensor product, so the joint success probability is the product w_i. The states for distinct labels are orthogonal. Consequently

$$\Pr[i \text{ and flag}] = \frac{w_i}{m}, \qquad \Pr[\text{flag}] = \frac{W}{m}, \qquad \Pr[i \mid \text{flag}] = \frac{w_i}{W}. \tag{6}$$

Until the algorithm lists $i^\star$, its weight exceeds $1/2$, so the flag probability W/m exceeds $1/(2m)$. Amplify the flag with this threshold and failure tolerance ε. On success append the measured label to the list. Stop on failure or after T draws. The finite schedule costs $O(\sqrt{m}\log(1/\varepsilon))$ calls to the preparation and its inverse on every branch. Grover rotations preserve the direction of the flagged component, so every successful trial has *exactly* the label distribution in (6) [9; 10; 5, Lemma 3.1].[4]

Thus at any history before the winner is listed, the algorithm stops with probability at most ε and otherwise has precisely the ideal next-pivot law. Coupling at each such history gives

$$\Pr[i^\star \text{ absent from the list}] \leq 2m2^{-T} + T\varepsilon. \tag{7}$$

Rebuilding at most T comparisons per preparation for at most T stages costs $O(c\sqrt{m}\,T^2 \log T)$ queries on every input.

3.3 Verifying the listed columns

For each listed column, decode a fresh address with error at most ε on promised inputs and use bounded quantum search to find any of its K tests that fails. Accept if at least one search returns no defect. When a test fails, a uniformly chosen test detects a defect with probability at least $1/K$. Using this threshold, the search costs $O(\ell\sqrt{K}\log(1/\varepsilon))$ queries per column and misses an existing defect with probability at most ε.

[3] In the worked exposition, §3.1, weights $(1, 0, 1/2, 1/4)$ give pivot probabilities $(4/7, 0, 2/7, 1/7)$.

[4] The rotation calculation explains both the unchanged conditional law and the bounded schedule. The formal quantum proof uses this exact measured-label law. Identifying the successful state of a designated physical output register is stronger; the retained-register argument and formal correspondence distinguish these assertions.

On a one-input, listing the winner and decoding its address correctly suffice for acceptance, since all its tests pass. Rejection therefore has probability at most (7) plus ε. On a zero-input, even a column with false transcripts could be selected in every pairwise comparison, since the comparison algorithm never reads a transcript. But for each listed column at least one test fails at *every* address (§2). Conditional on the list, fresh searches give false-acceptance probability at most $T\varepsilon$. Both errors are below $1/3$.

The cost of checking the list is $O(T(c + \ell\sqrt{K}\log T))$, including fresh address decoding. With $T = O(\ell)$, $c = O_k(\ell \log \ell)$ and $K = O_k(m\ell^3)$, this and the tournament cost sum to at most $C_k\sqrt{m}\,\ell^6$. The quantum algorithm can decode the address before reading a pointer. How often can a classical tree read even one bit of the winning certificate when it must first supply the entire address?

4. A randomized lower bound from the cell address

4.1 Planting one certificate

For the **dummy input** $Y^{(0)}$, choose the row J_i of the sole zero in X_i independently and uniformly in each column. Draw the address inputs independently of one another and of the zero positions, using a common law μ on the promise of P. We will choose μ to make even one address bit hard to predict. Set every cell bit to zero. Every column loses. Choose I uniformly and independently of the entire dummy input. For the **planted input** $Y^{(1)}$, set $X_I[J_I] = 1$. In B_{I,s_I} write the gate bits for all r correct circuit evaluations and the row numbers J_j for $j \neq I$. Column I wins.

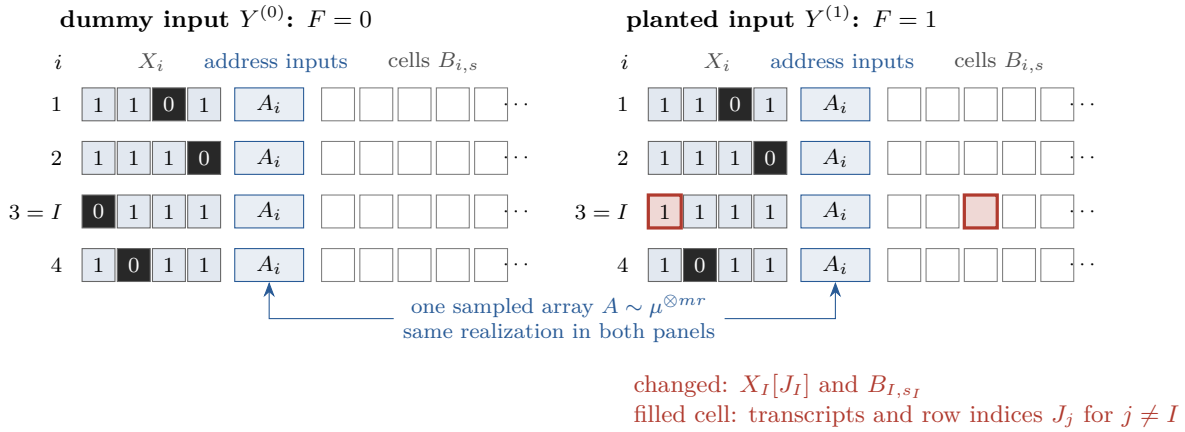


Figure 2: Coupled inputs with $I = 3$. Both panels use the same strings $A_{i,h}$. The only bits that can differ are $X_I[J_I]$ and the gate and pointer bits of B_{I,s_I}.

Run a deterministic decision tree on both inputs. The sequences of queried bit indices and replies coincide up to the first query to $X_I[J_I]$ or to a bit of B_{I,s_I}. Thus the acceptance gap is at most the sum of the probabilities that the tree queries these locations on the dummy input [2, Lemma 6; 11, § 5.1]. The dummy transcript is independent of I. For the cell, we distinguish runs with many queries to the strings $A_{I,h}$ from runs with few such queries that nevertheless include a query to B_{I,s_I}.

For any integer $1 \le L \le m$ with $L < R(P)$, choose μ by minimax so that every deterministic depth-L tree computes P with success at most $\beta = 2/3$ [12]. Give the tree on the coupled inputs $G = \lfloor mL/100 \rfloor$ queries. First bound the probability of querying the erased zero.

4.2 Searching for the erased zero

Let Z count the columns i in which the tree queries the unique zero of X_i on the dummy input. By independence, the probability that I is among these columns is $\mathbb{E}Z/m$. Before the first

query to a column's zero, its row is uniform among the unqueried rows even conditional on the transcript. Until the tree has queried half the rows of that column, the probability of finding the zero on its next row query there is at most $2/m$. The expected number of zeros found this way is at most $2G/m$. In each other column where the tree finds a zero it uses at least $m/2$ queries, so there are at most $2G/m$ further zeros. Hence

$$\Pr[\text{query } X_I[J_I]] \le \frac{4G}{m^2}. \tag{8}$$

4.3 Predicting the cell address

The index of any query to a bit of $B_{i,s}$ includes s. With two address bits, querying the first domain bit of $B_{i,10}$ proposes the two values $(1, 0)$ before the bit is returned. If $s = s_i$, the index contains the correct r values of P for column i. Call a column *heavy* if the tree makes more than L queries to its address inputs on the full dummy run, counting repetitions, and *light* otherwise. There are at most G/L heavy columns on any run. Since I is independent of that run,

$$\Pr[I \text{ heavy}] \le \frac{G}{mL}.$$

In a light column the tree has made at most L queries to those r inputs. We cannot apply a direct-product bound conditional on the column being light. The tree may use earlier replies to decide whether to query the column again, so this conditioning can change the input distribution.[5]

Fix a column i and query time $\tau \le G$. Use the predictor's r unknown strings as $A_{i,1}, \dots, A_{i,r}$. Independently sample all zero rows and the other columns' address strings as on a dummy input, and set all cell bits to zero. Simulate the tree's queries before τ, querying the unknown strings only for requested bits of $A_{i,h}$. Stop before the $(L+1)$-st such query in total. Inspect the bit index the tree would query at time τ. If it specifies a bit of $B_{i,s}$, output s without reading that bit. For any other index, early termination of the tree, or a simulation already stopped by the cap, output a fixed r-bit string. The worked exposition, Appendix C, "Two clocks in a capped simulation" separates the tree's query time from the predictor's target-read count.

The total cap gives the predictor at most L queries to *each* target input, on every branch. Under $\mu^{\otimes r}$ its success probability is therefore at most β^r by the separate-cap direct-product bound. This bound allows adaptive interleaving and side information and coins jointly independent of the tuple [13, § 3; 5, Lemma 4.1].[6]

If i is light at the end of the run, the simulation uses at most L target queries through time τ. If the tree queries B_{i,s_i} at that time, the predictor outputs s_i. Correct default guesses can also make the predictor succeed; only this event inclusion is needed for the joint bound

$$\Pr[i \text{ light and query } \tau \text{ is in } B_{i,s_i}] \;\le\; \Pr[\text{predictor outputs } s_i] \;\le\; \beta^r. \tag{9}$$

For each fixed column there are at most G opportunities to make this prediction. Average over the independent choice of I and add the chance of a heavy column to obtain

$$\Pr[\text{query } B_{I,s_I}] \le \frac{G}{mL} + \frac{1}{m}\sum_{i=1}^{m} G\beta^r = \frac{G}{mL} + G\beta^r. \tag{10}$$

[5] For AND on the equally likely inputs $01, 10, 11$, one query has optimal success $2/3$. Reading the first bit and stopping after one read only when it is zero gives success one conditional on using just one read. The joint-event calculation explains why this does not contradict the one-query bound.

[6] Fix the independent coins and side information. Given the queried indices and replies, the r strings remain independent. For each string take the greatest probability of predicting its value under P with its remaining queries. A query to one input leaves the other factors fixed and cannot increase the chosen factor in expectation, since that query was already an available next step. The product gives the separate-cap bound.

4.4 The randomized bound

Combining (8) and (10), the gap between the acceptance probabilities on the two laws is at most

$$\frac{4G}{m^2} + \frac{G}{mL} + G(2/3)^r < \frac{1}{3}. \tag{11}$$

Here $G \le mL/100 \le m^2/100$ and $r = 20 \log_2 m$ gives $G(2/3)^r \le Gm^{-10}$. Averaging over coins gives (11) for a randomized tree, while correctness with error at most $1/3$ on both supports would force a gap of at least $1/3$. Thus any bounded-error randomized algorithm for this total function requires more than $\lfloor mL/100 \rfloor$ randomized queries. For (3) at fixed k, the erased-zero term is $O(L/m) = o(1)$ and $G(2/3)^r$ also tends to zero. The restriction $G = O(mL)$ comes from bounding the fraction of heavy columns by $G/(mL)$.

5. Forrelation and the optimal exponent

We require a partial function P with a low-query quantum algorithm on its promise, a nearly linear exact circuit for checking its domain and value, and high randomized query complexity. The Walsh–Hadamard form of k-fold Forrelation supplies all three properties [3; 14]. Let H be the normalized Walsh–Hadamard matrix of order m, $u = m^{-1/2}(1, \dots, 1)^{\mathsf{T}}$, and D_z the diagonal matrix of a sign vector. For k such vectors of length m, put

$$\Phi = u^{\mathsf{T}} D_{z_1} H D_{z_2} H \cdots H D_{z_k} u, \qquad \delta = 2^{-5k}.$$

Set $P = 1$ when $\Phi \ge \delta$, $P = 0$ when $|\Phi| \le \delta/2$, and leave P undefined elsewhere. Applying the unitary product to u and projecting onto u succeeds with probability Φ^2. For fixed k the two promised cases have a constant gap, giving the claimed decoder by repetition with trial registers retained. The projection probability depends only on $|\Phi|$, whereas an input with $\Phi \le -\delta$ lies outside the promise. For the exact circuit, compute the signed numerator by $k-1$ unnormalized fast Walsh–Hadamard transforms on integers of $O_k(\ell)$ bits and test the two defining inequalities after clearing denominators. This yields both domain and value bits in $O_k(m\ell^2)$ fan-in-two gates [5, Lemma 2.1]. The lower bound of Bansal and Sinha [3, Corollary 1.4] gives (3). Fixing this circuit in the cells defines $F_{k,m}$.

For sufficiently large m, the floor in (3) gives $L \ge m^{1-1/k}/(2\ell^2)$ and $\lfloor mL/100 \rfloor \ge mL/200$, proving the randomized inequality of (1). Now fix $0 \le \alpha < 4$. Choose $k \ge 2$ with $k > 2/(4-\alpha)$ and hold it fixed as m grows through powers of four. By (1),

$$\frac{R(F_{k,m})}{(1+Q(F_{k,m}))^{\alpha}} \;\ge\; \frac{1}{400(1+C_k)^{\alpha}} \, \frac{m^{2-1/k-\alpha/2}}{(\log_2 m)^{2+6\alpha}} \;\longrightarrow\; \infty.$$

Thus for each exponent below four some fixed k gives a counterexample, proving optimality of the quartic upper bound. The quantum algorithm uses $\widetilde{O}_k(\sqrt{m})$ queries to sample the list and as many again to verify it. For a uniformly chosen winning column, the randomized lower bound is of order mL, where L is already a lower bound for computing one address bit.

References

[1] S. Aaronson, S. Ben-David, R. Kothari, S. Rao and A. Tal. *Degree vs. approximate degree and quantum implications of Huang's sensitivity theorem.* Proceedings of the 53rd annual ACM symposium on theory of computing (STOC 2021), 1330–1342, 2021, arXiv:2010.12629.

[2] S. Aaronson, S. Ben-David and R. Kothari. *Separations in query complexity using cheat sheets.* Proceedings of the 48th annual ACM symposium on theory of computing (STOC 2016), 863–876, 2016, arXiv:1511.01937v2.

[3] N. Bansal and M. Sinha. *k-Forrelation optimally separates quantum and classical query complexity.* Proceedings of the 53rd annual ACM symposium on theory of computing (STOC 2021), 1303–1316, 2021, arXiv:2008.07003v3.

[4] A. A. Sherstov, A. A. Storozhenko and P. Wu. *An optimal separation of randomized and quantum query complexity.* SIAM Journal on Computing, 52 (2), 525–567, 2023, arXiv:2008.10223v4, 29 January 2023.

[5] OpenAI. *A nearly quartic separation between randomized and quantum query complexity.* 2026, Preprint dated 5 October 2026. https://github.com/openai/math/blob/4eaee875/preprints/A-Nearly-Quartic-Separation-Between-Randomized-and-Quantum-Query-Complexity-October-5-2026/quartic-query-separation.pdf.

[6] M. Göös, T. Pitassi and T. Watson. *Deterministic communication vs. partition number.* SIAM Journal on Computing, 47 (6), 2435–2450, 2018, preliminary version in FOCS 2015.

[7] A. Ambainis, K. Balodis, A. Belovs, T. Lee, M. Santha and J. Smotrovs. *Separations in query complexity based on pointer functions.* Journal of the ACM, 64 (5), 32:1–32:24, 2017, arXiv:1506.04719v3, 26 October 2015.

[8] A. Ambainis, J. Iraids and M. Kokainis. *Near-optimal separations of certificate complexity from randomized and quantum query complexity.* 2026, arXiv:2609.11664v3, Section 6.1. https://arxiv.org/abs/2609.11664v3.

[9] M. Boyer, G. Brassard, P. Høyer and A. Tapp. *Tight bounds on quantum searching.* Fortschritte der Physik, 46 (4–5), 493–505, 1998, arXiv:quant-ph/9605034v1, Section 4.

[10] G. Brassard, P. Høyer, M. Mosca and A. Tapp. *Quantum amplitude amplification and estimation.* Quantum computation and quantum information, Contemporary mathematics, 305, 53–74, American Mathematical Society, 2002, arXiv:quant-ph/0005055v1.

[11] S. Ben-David. *A super-Grover separation between randomized and quantum query complexities.* 2015, arXiv:1506.08106.

[12] A. C.-C. Yao. *Probabilistic computations: Toward a unified measure of complexity.* Proceedings of the 18th annual symposium on foundations of computer science (FOCS 1977), 222–227, 1977.

[13] A. Drucker. *Improved direct product theorems for randomized query complexity.* Computational Complexity, 21 (2), 197–244, 2012, arXiv:1005.0644v2.

[14] S. Aaronson and A. Ambainis. *Forrelation: A problem that optimally separates quantum from classical computing.* SIAM Journal on Computing, 47 (3), 982–1038, 2018, arXiv:1411.5729v1, Section 3.2.

Further reading. The exponent proof above chooses k for each proposed $\alpha < 4$ and then keeps k fixed. The worked exponent calculation follows this order. The full reconstruction separately tracks the dependence on k to obtain a single family with growing k; substituting a growing parameter into (1) alone would not justify that conclusion. The theorem index links the formal statements and Lean proofs. Will Cook directed the reconstruction. AI systems drafted and revised the manuscripts and Lean development.